

Caso 1: Aumento de tamaño de punto (GL_POINTS)

Línea de código: `glPointSize(10.0f);`

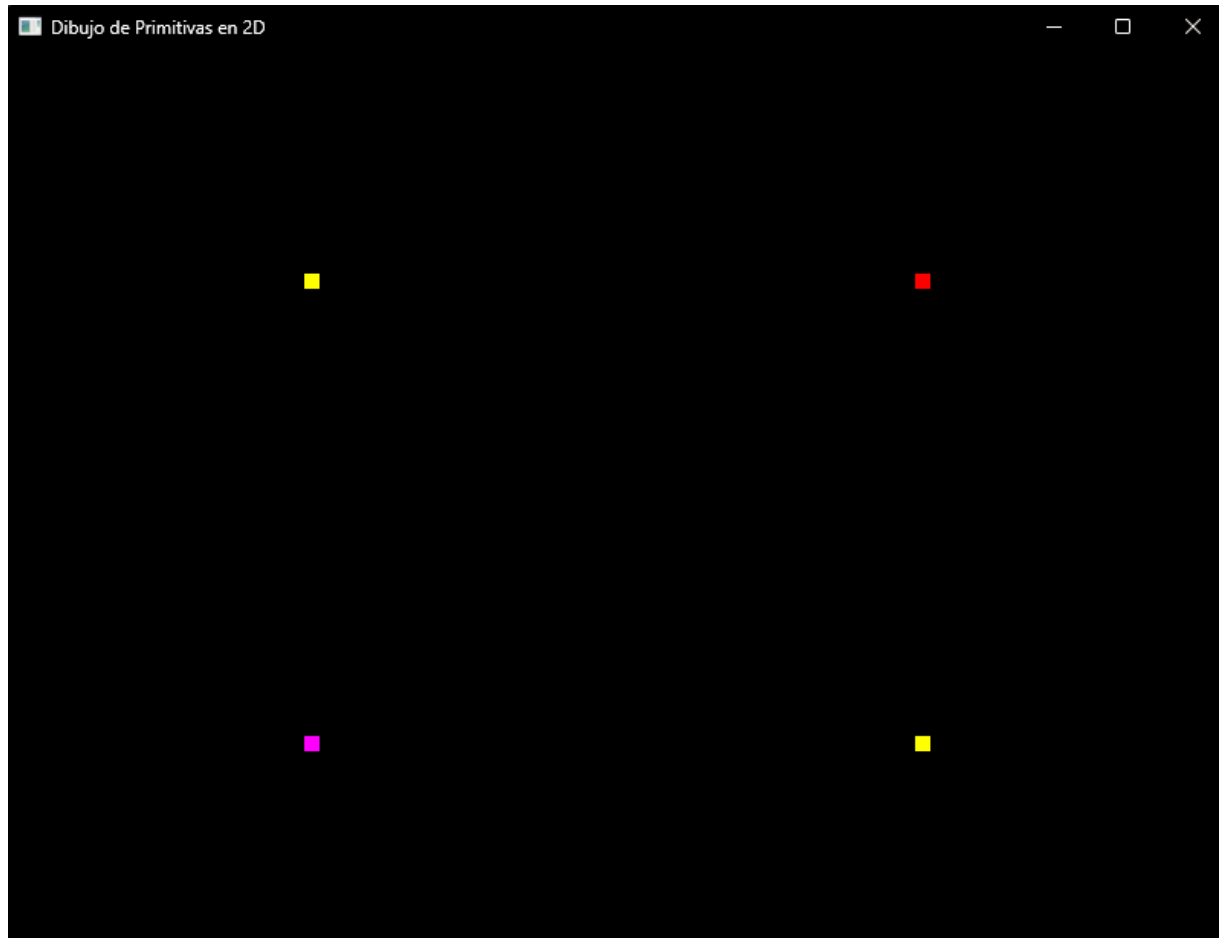
Descripción: Se modificó el tamaño del punto en pantalla utilizando `glPointSize` para visualizar de forma clara el primer vértice cargado en el VBO.



Caso 2: Renderizado de múltiples vértices con atributos de color

Línea de código: `glDrawArrays(GL_POINTS, 0, 4);`

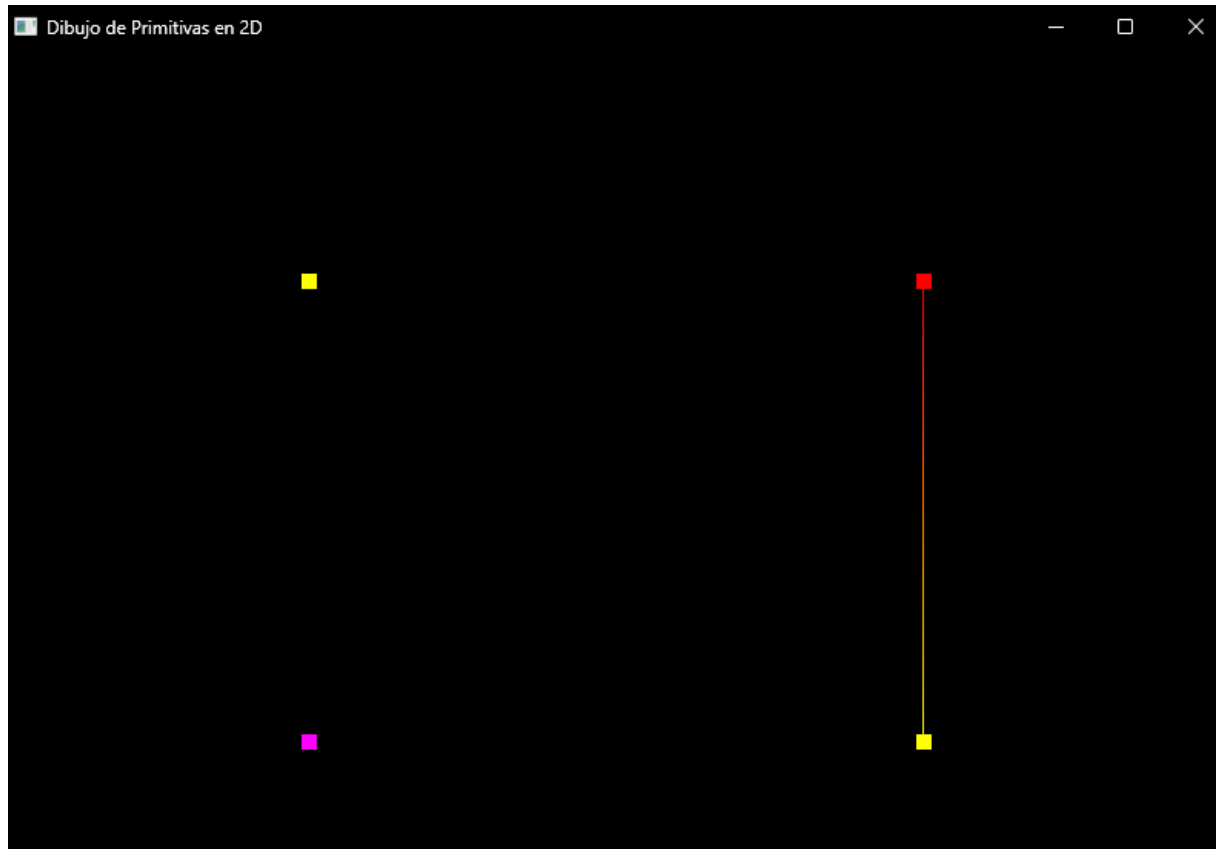
Descripción: Se aumentó el conteo a 4 vértices para dibujar todos los puntos definidos en el arreglo, mostrando la interpolación de color asociada a cada posición.



Caso 3: Renderizado de segmentos de línea (GL_LINES)

Línea de código: `glDrawArrays(GL_LINES, 0, 2);`

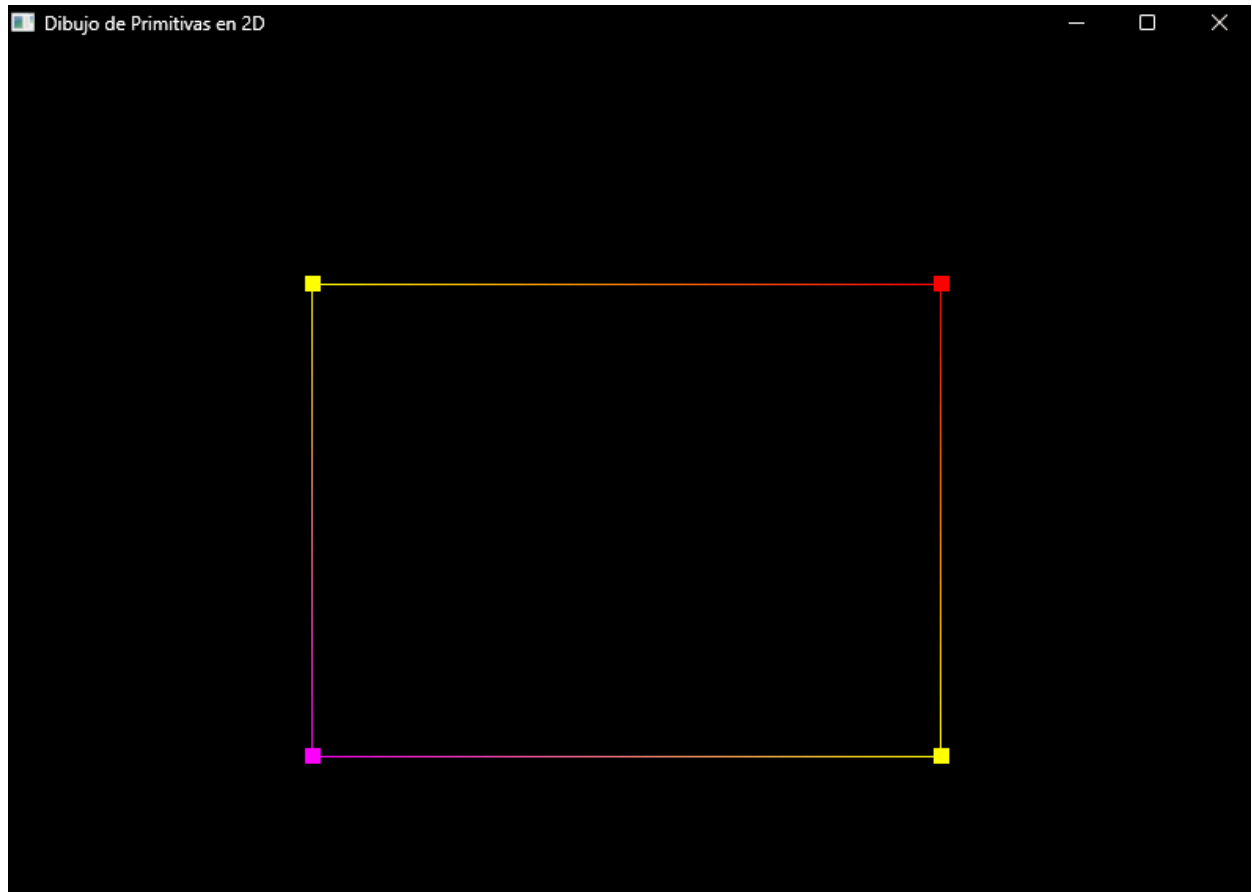
Descripción: Se utilizó la primitiva `GL_LINES` para conectar los primeros dos vértices mediante una arista, observando el gradiente de color entre ambos puntos.



Caso 4: Contorno cerrado (GL_LINE_LOOP)

Línea de código: `glDrawArrays(GL_LINE_LOOP, 0, 4);`

Descripción: Se conectaron los cuatro vértices en secuencia cerrando la figura hacia el vértice inicial, formando el perímetro cuadrado de la primitiva.



Caso 5: Triángulo con interpolación de color (GL_TRIANGLES)

Línea de código: `glDrawArrays(GL_TRIANGLES, 0, 3);`

Descripción: Se renderizó un triángulo sólido a partir de los primeros 3 vértices, apreciando la rasterización y el degradado calculado por el Fragment Shader.



Caso 6: Cuadrilátero mediante índices (glDrawElements)

Línea de código: `glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);`

Descripción: Se dibujó el cuadrado completo conformado por dos triángulos utilizando el Element Buffer Object (EBO) para indexar los vértices sin duplicar información en memoria.

